

Background for HDC Thesis Research

Hyperdimensional Computing for Molecular Property Prediction

Information Fusion & Attention Mechanisms

Advisor Meeting — 2026-04-13 Gustavo Vazquez

Students

Alexia Aurrecochea

Emiliano Fau

Roadmap

This presentation covers **all the background** needed for two thesis projects:

1.  **Molecular Representations** — How molecules become data
2.  **HDC Foundations** — The computing paradigm
3.  **HDC for Molecules** — MoleHD & HDBind
4.  **Information Fusion** — Combining multiple views
5.  **Attention Mechanisms** — Selective focus in HDC
6.  **Topological Data Analysis** — Shape-based features
7.  **Thesis Directions** — Your research projects

Part I

Molecular Representations for Machine Learning

Why Molecular Property Prediction?

Drug discovery is one of the most expensive human endeavors:

- 💰 Average cost to develop one drug: **\$2.6 billion**
- ⌚ Average timeline: **10-15 years**
- ❌ Clinical trial failure rate: **~90%**

Key question: Can we predict molecular properties *computationally* before expensive lab experiments?

Key idea. If we can predict whether a molecule is toxic, crosses the blood-brain barrier, or binds to a protein target — *in silico* — we can dramatically reduce cost and time.

From Molecules to Data

A molecule must be converted into a **numerical representation** for ML:

Representation	Type	Example
SMILES	String	<chem>CC(=O)OC1=CC=CC=C1C(=O)O</chem>
Molecular Graph	Graph	Atoms = nodes, bonds = edges
3D Conformer	Point cloud	(x, y, z) per atom
Fingerprint	Binary vector	[0, 1, 1, 0, 0, 1, ...]
Descriptors	Real vector	[MW=180.2, LogP=1.2, ...]

Remark. Each representation captures *different aspects* of the molecule. This is the key motivation for fusion.

SMILES: Simplified Molecular Input Line Entry System

SMILES encodes molecular structure as a **text string**:

- **Atoms:** C, N, O, S, ...
- **Bonds:** - single, = double, # triple
- **Branches:** parentheses ()
- **Rings:** digit pairs

Examples:

Molecule	SMILES
Ethanol	<chem>CCO</chem>
Benzene	<chem>c1ccccc1</chem>
Aspirin	<chem>CC(=O)OC1=CC=CC=C1C(=O)O</chem>
Caffeine	<chem>CN1C=NC2=C1C(=O)N(C)C(=O)N2C</chem>

Molecular Graphs

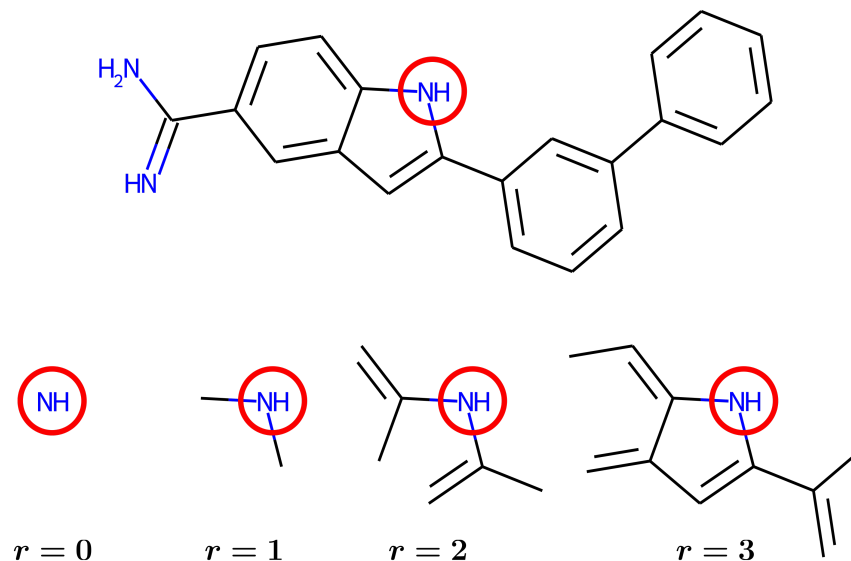
A molecule is naturally a **graph** $G = (V, E)$:

- **Nodes** V : atoms (with features: element, charge, hybridization)
- **Edges** E : bonds (with features: single, double, aromatic)

This is the basis for **Graph Neural Networks (GNNs)** in chemistry.

Key idea. A molecular graph captures *connectivity* (which atoms are bonded to which) but not the 3D spatial arrangement.

ECFP: Extended-Connectivity Fingerprints



Also known as **Morgan fingerprints**, ECFP encodes the local *neighborhood* around each atom:

Algorithm (radius r , length n):

1. Initialize: each atom gets an identifier based on its properties
2. For $i \equiv 1, \dots, r$: update each atom's identifier by hashing its current ID with its

What ECFP Captures

ECFP captures **local chemical neighborhoods** — circular subgraphs around atoms:

- **Radius 0:** Individual atom types
- **Radius 1:** Atom + immediate neighbors
- **Radius 2:** Atom + neighbors + neighbors' neighbors

ECFP: Strengths and Weaknesses

Strengths: Captures functional groups, pharmacophores, local reactivity

Weaknesses: Cannot capture:

- Long-range through-space interactions
- 3D shape and conformation
- Global molecular topology (cavities, pockets)

Remark. ECFP is the most widely used molecular representation in drug discovery. HDBind uses it directly as a hypervector encoding.

MoleculeNet: The Standard Benchmark

MoleculeNet (Wu et al., 2018) provides standardized datasets for molecular ML:

Dataset	Molecules	Tasks	Property Type
BBBP	2,039	1	Blood-brain barrier penetration
Tox21	7,831	12	Toxicity endpoints
ClinTox	1,478	2	Clinical trial toxicity
HIV	41,127	1	HIV replication inhibition
BACE	1,513	1	β -secretase 1 inhibition
SIDER	1,427	27	Drug side effects

Self-Supervised Molecular Representations

Modern approaches learn molecular representations from **unlabeled data**:

MolFormer (Ross et al., 2022):

- Transformer-based language model
- Pre-trained on **1.1 billion** SMILES strings
- Produces 768-dimensional embeddings
- Captures distributional patterns across chemical space

Self-Supervised: MolCLR

MolCLR (Wang et al., 2022):

- Graph neural network with contrastive learning
- Learns representations via augmentation (atom masking, bond deletion, subgraph removal)
- Captures graph structural patterns

Key idea. These models encode "knowledge" about chemistry learned from massive molecular datasets — knowledge that can be transferred to downstream tasks.

Part II

Foundations of Hyperdimensional Computing

The AI Paradigm Landscape

Two classical approaches to AI:

	Symbolic AI (GOFAI)	Connectionist AI (Neural Nets)
Strength	Structured reasoning, compositionality	Pattern recognition, learning from data
Weakness	Brittle, no learning, requires manual rules	No structure, "black box", expensive
Representations	Symbols with no inherent similarity	Distributed vectors, but no compositionality

Key idea. HDC sits at the **intersection**: it uses distributed vector representations (like neural nets) but supports structured composition (like symbolic AI).

References: Kanerva (2009), Kleyko et al. (2022-2023)

What is Hyperdimensional Computing?

Definition. Hyperdimensional Computing (HDC), also called Vector Symbolic Architectures (VSA), is a computing paradigm that represents information as high-dimensional vectors (hypervectors) and performs reasoning through simple algebraic operations.

Key characteristics:

- **Dimension:** $D \geq 10,000$ (typically 10k)
- **Operations:** Element-wise multiplication, addition, permutation
- **Learning:** Single-pass (no gradient descent)
- **Inference:** Nearest-neighbor in cosine space
- **Model size:** Often < 1 MB

The Blessing of Dimensionality

In **low dimensions**, random vectors frequently collide:

- In $\mathbb{R}^3$: random vectors are often similar

In **high dimensions** ($D \geq 10,000$), a remarkable phenomenon occurs:

- Random vectors are **almost always nearly orthogonal**
- The cosine similarity concentrates around **0**

Theorem (Concentration of Measure). For two random i.i.d. vectors $\mathbf{a}, \mathbf{b} \in \{-1, +1\}^D$, as $D \rightarrow \infty$:

$$\cos(\mathbf{a}, \mathbf{b}) \rightarrow 0 \quad \text{with high probability}$$

The standard deviation shrinks as $\sim 1/\sqrt{D}$.

For $D = 10,000$: standard deviation ≈ 0.01 .

Quasi-Orthogonality: The Key Property

With $D = 10,000$:

- **Any two random hypervectors** are nearly orthogonal ($\cos \approx 0$)
- This means we can generate an **exponential number** of distinct, non-interfering representations
- For $D = 10,000$ binary vectors: $2^{10,000}$ possible vectors!

Quasi-Orthogonality: The Key Property

Implication for information storage:

- Each hypervector has a vast "private neighborhood"
- Noise corruption stays within this neighborhood
- Information degrades **gracefully**, never catastrophically

Key idea. High dimensionality is not a curse — it is a **blessing** that gives us near-infinite capacity for distinct representations.

Hypervector Types

Three main types of hypervectors:

Type	Domain	Example
Binary	$\{0, 1\}^D$	[0, 1, 1, 0, 1, 0, ...]
Bipolar	$\{-1, +1\}^D$	[-1, +1, +1, -1, +1, ...]
Real-valued	$\mathbb{R}^D$	[0.3, -0.7, 0.1, ...]

Definition. A hypervector $\mathbf{h} \in \mathcal{H}^D$ is a vector in a high-dimensional space ($D \geq 10,000$) used to represent an entity, concept, or data point.

Generation: Hypervectors are typically generated by sampling each component **i.i.d.** from a uniform distribution.

For bipolar: $h_i \sim \text{Uniform}(\{-1, +1\})$

The Three Core Operations

HDC's algebra consists of three operations:

Operation	Symbol	Implementation	Similarity Effect
Binding	$\otimes$	Element-wise multiplication	Result is dissimilar to both inputs
Bundling	$\oplus$	Element-wise addition	Result is similar to all inputs
Permutation	Π	Cyclic rotation	Result is dissimilar to input

These three operations are sufficient to build complex data structures:

- Records, sequences, sets, graphs, trees

Key idea. With just three simple operations, HDC can represent arbitrarily complex structured data.

Operation 1: Binding ($\otimes$)

Element-wise (Hadamard) product of two hypervectors:

$$\mathbf{c} = \mathbf{a} \otimes \mathbf{b} = \langle a_1 b_1, a_2 b_2, \dots, a_D b_D \rangle$$

Properties:

-  Result is **quasi-orthogonal** to both **a** and **b**
-  Commutative: $\mathbf{a} \otimes \mathbf{b} = \mathbf{b} \otimes \mathbf{a}$
-  Associative: $(\mathbf{a} \otimes \mathbf{b}) \otimes \mathbf{c} = \mathbf{a} \otimes (\mathbf{b} \otimes \mathbf{c})$
-  Self-inverse: $\mathbf{a} \otimes \mathbf{a} = \mathbf{1}$
-  Distributes over bundling

Definition. Binding creates an *association* between two concepts. The result represents "A paired with B" — a new entity distinct from both.

Binding: Intuition and Example

Semantic meaning: Binding says " X is associated with Y "

Example (bipolar, $D = 8$ for illustration):

$$\mathbf{color} = [+1, -1, +1, +1, -1, +1, -1, +1]$$

$$\mathbf{red} = [-1, +1, +1, -1, +1, -1, +1, -1]$$

$$\mathbf{color} \otimes \mathbf{red} = [-1, -1, +1, -1, -1, -1, -1, -1]$$

Verification: $\cos(\mathbf{color} \otimes \mathbf{red}, \mathbf{color}) = -0.25$ (dissimilar )

Unbinding (self-inverse property):

$$\mathbf{color} \otimes (\mathbf{color} \otimes \mathbf{red}) = \mathbf{red}$$

Remark. Binding is invertible: you can recover **red** by binding the result with **color** again.

Operation 2: Bundling ($\oplus$)

Element-wise addition (optionally followed by sign quantization):

$$\mathbf{c} = \mathbf{a} \oplus \mathbf{b} = \langle a_1 + b_1, a_2 + b_2, \dots, a_D + b_D \rangle$$

Properties:

-  Result is **similar** to all inputs: $\cos(\mathbf{a} \oplus \mathbf{b}, \mathbf{a}) > 0$
-  Commutative and associative
-  **NOT invertible**: once bundled, cannot separate components
-  Capacity: can bundle up to $\sim \sqrt{D}$ vectors reliably

Definition. Bundling creates a *superposition* — a single vector that represents a set of items. The result is "similar to all of them" simultaneously.

Bundling: Intuition and Example

Semantic meaning: Bundling says " X and Y and Z belong together"

Example (bipolar, $D = 8$):

$$\mathbf{a} = [+1, -1, +1, +1, -1, +1, -1, +1]$$

$$\mathbf{b} = [-1, +1, +1, -1, +1, -1, +1, -1]$$

$$\mathbf{c} = [+1, +1, -1, +1, -1, +1, +1, -1]$$

$$\mathbf{a} \oplus \mathbf{b} \oplus \mathbf{c} = [+1, +1, +1, +1, -1, +1, +1, -1]$$

(after majority-rule quantization)

Verification: $\cos(\text{bundle}, \mathbf{a}) = 0.5 > 0$  (similar to $\mathbf{a}$)

Key idea. Bundling is how HDC builds **class prototypes** — by adding up all training examples that belong to the same class.

Operation 3: Permutation (Π)

Cyclic rotation by k positions:

$$\Pi^k(\mathbf{a}) = \langle a_{D-k+1}, \dots, a_D, a_1, \dots, a_{D-k} \rangle$$

Properties:

-  Result is quasi-orthogonal to input: $\cos(\Pi(\mathbf{a}), \mathbf{a}) \approx 0$
-  Preserves distances: $\cos(\Pi(\mathbf{a}), \Pi(\mathbf{b})) = \cos(\mathbf{a}, \mathbf{b})$
-  Invertible: $\Pi^{-k}(\Pi^k(\mathbf{a})) = \mathbf{a}$

Definition. Permutation encodes *position* or *order*. By applying k rotations, we mark an element as being in position k .

Use case: Encoding sequences:

$$\mathbf{h}_{\text{seq}} = \Pi^2(\mathbf{h}_A) + \Pi^1(\mathbf{h}_B) + \Pi^0(\mathbf{h}_C)$$

Building Complex Structures

Combining operations enables structured representation:

Key-value pair (record field):

$$\mathbf{h}_{\text{pair}} = \mathbf{key} \otimes \mathbf{value}$$

Record (multiple key-value pairs):

$$\mathbf{h}_{\text{record}} = \bigoplus_i (\mathbf{key}_i \otimes \mathbf{value}_i)$$

Example — encoding a molecule's properties:

$$\mathbf{h}_{\text{mol}} = (\mathbf{k}_{\text{MW}} \otimes \mathbf{v}_{180}) \oplus (\mathbf{k}_{\text{LogP}} \otimes \mathbf{v}_{1.2}) \oplus (\mathbf{k}_{\text{rings}} \otimes \mathbf{v}_2)$$

Key idea. The algebraic framework lets us build arbitrarily complex data structures — records, sequences, trees, graphs — all stored in a single D -dimensional vector.

Item Memory (Codebook)

Definition. The **Item Memory** (IM) is a lookup table that maps each atomic symbol or token to a random seed hypervector.

For a vocabulary $\{t_1, t_2, \dots, t_V\}$:

$$\text{IM} = \{\mathbf{B}_1, \mathbf{B}_2, \dots, \mathbf{B}_V\}$$

where each $\mathbf{B}_i \in \{-1, +1\}^D$ is generated **randomly** and **independently**.

Guarantees (from concentration of measure):

- All base vectors are mutually quasi-orthogonal
- $\cos(\mathbf{B}_i, \mathbf{B}_j) \approx 0$ for $i \neq j$

Remark. No training is needed to create the Item Memory — randomness guarantees separation.

Encoding: From Data to Hypervectors

General encoding pipeline:

1. **Tokenize** the input data into discrete tokens
2. **Look up** each token in the Item Memory → one HV per token
3. **Combine** token HVs using binding, bundling, permutation
4. **Quantize** (binarize/bipolarize) the result

Encoding: From Data to Hypervectors

For a sequence $[t_1, t_2, \dots, t_n]$ (n-gram encoding):

$$\mathbf{h}_{\text{encoded}} = \sigma \left(\sum_{i=1}^n \Pi^{n-i}(\mathbf{B}_{t_i}) \right)$$

where $\sigma(\cdot) = \text{sign}(\cdot)$ is the bipolarization function.

This gives each token a **position-dependent** representation via permutation, then aggregates via bundling.

Training: Single-Pass Learning

HDC training builds **class prototypes** by bundling:

For class k : aggregate all training hypervectors of that class:

$$\mathbf{C}_k = \sum_{i:y_i=k} \mathbf{h}_i$$

That's it! No gradient descent, no loss function, no backpropagation.

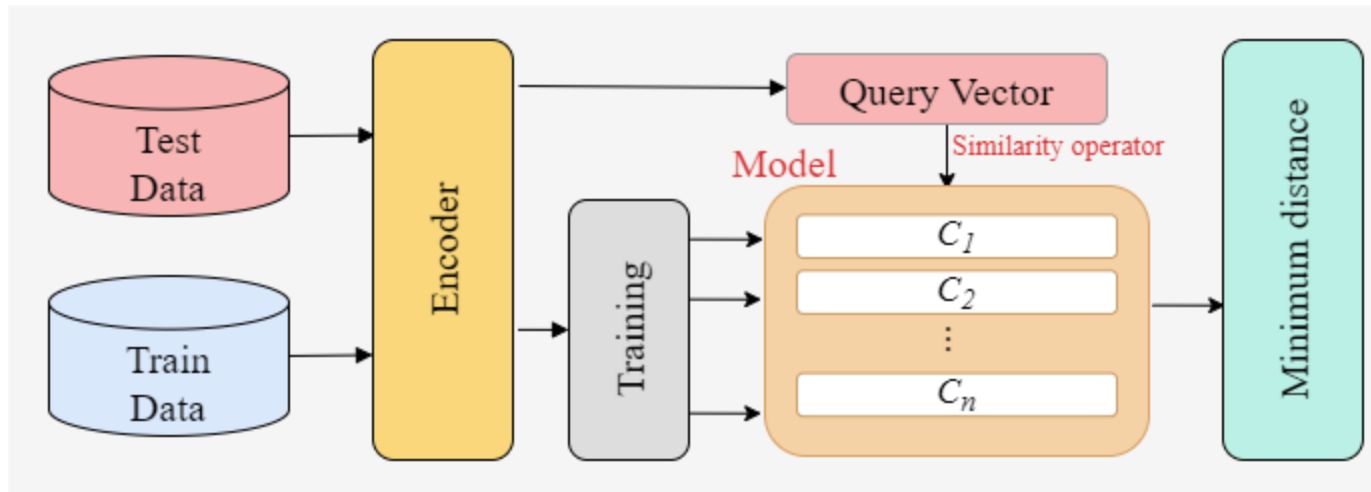
Properties of training:

- ⚡ **One-pass**: processes each sample exactly once
- 🔄 **Incremental**: new samples can be added by simple addition
- 💾 **Memory**: only K prototype vectors (one per class)

Training: Single-Pass Learning

Key idea. HDC training is fundamentally different from deep learning. It is a simple accumulation: "add all examples of class k together." The resulting prototype represents the *average* of that class.

Training: Single-Pass Learning



Remark. The entire pipeline uses only: random generation, element-wise operations, and dot products. No matrix multiplications, no gradients.

Retraining: Error-Driven Refinement

Single-pass accuracy can be improved with **iterative retraining**:

Algorithm:

```
For each epoch:
  For each training sample (h_i, y_i):
     $\hat{y} = \operatorname{argmax}_k \cos(h_i, C_k)$  # predict
    if  $\hat{y} \neq y_i$ : # misclassified
       $C_{\{y_i\}} = C_{\{y_i\}} + h_i$  # add to correct class
       $C_{\{\hat{y}\}} = C_{\{\hat{y}\}} - h_i$  # subtract from wrong class
```

\n---\n\n# Retraining Dynamics\n\nThis **moves prototypes** to better separate the classes.

Remark. Retraining typically converges in 20-50 epochs. It is much faster than backpropagation because it uses only additions and subtractions — no matrix multiplications or chain rule.

Inference: Nearest-Neighbor Search

Given a query hypervector $\mathbf{h}_q$:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} \cos(\mathbf{h}_q, \mathbf{C}_k)$$

where:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Confidence score (for binary classification):

$$\eta = \frac{1}{2} + \frac{1}{4} (\cos(\mathbf{h}_q, \mathbf{C}_+) - \cos(\mathbf{h}_q, \mathbf{C}_-))$$

Key idea. Inference in HDC is a **dot product** — massively parallelizable and hardware-friendly. This is why HDC achieves extreme energy efficiency on FPGAs and neuromorphic chips

Part III

HDC for Molecular Classification

MoleHD: Brain-Inspired Drug Discovery

MoleHD (Ma et al., 2022) — the first HDC method for molecular property prediction.

Core idea: Treat a SMILES string as a *sentence* and apply HDC encoding:

1. **Tokenize** the SMILES string
2. **Encode** tokens into hypervectors using Item Memory
3. **Combine** via n-gram encoding (permutation + bundling)
4. **Train** class prototypes
5. **Classify** new molecules

Key idea. MoleHD treats molecules like text — leveraging the sequential patterns in SMILES to encode chemical substructures.

MoleHD: Two Tokenization Strategies

1. Character-level (MoleHD-char):

Each SMILES character is a token.

CCO → [C, C, O] (3 tokens)

2. SMILES Pair Encoding (MoleHD-PE):

Data-driven subword tokenization that learns meaningful substructures.

CC(=O)OC1=CC=CC=C1 → [CC, (=O), OC1, =CC, =CC, =C1]

Which is better?

- PE captures chemically meaningful substructures
- Character-level is simpler but misses patterns

Remark. SPE (SMILES Pair Encoding) is analogous to Byte Pair Encoding (BPE) used in NLP transformers — a familiar concept for CS students.

MoleHD: Encoding Pipeline

For a tokenized sample $[t_1, t_2, \dots, t_n]$:

Step 1: Look up base hypervectors from Item Memory:

$$\mathbf{B}_{t_1}, \mathbf{B}_{t_2}, \dots, \mathbf{B}_{t_n}$$

Step 2: Apply positional encoding via permutation:

$$\mathbf{h}_i = \Pi^{n-i}(\mathbf{B}_{t_i})$$

Step 3: Aggregate via bundling (element-wise sum):

$$\mathbf{S} = \sum_{i=1}^n \Pi^{n-i}(\mathbf{B}_{t_i})$$

Step 4: Bipolarize:

$$\mathbf{h}_{\text{mol}} = \text{sign}(\mathbf{S})$$

MoleHD: Results

Performance (ROC-AUC, $D = 10,000$):

Dataset	MoleHD-PE	RF	SVM	GCN
BBBP	87.9	71.2	72.9	87.7
ClinTox	98.7	71.3	93.1	90.7
SIDER	61.0	68.3	62.8	60.9

MoleHD: Model Characteristics

Model characteristics:

-  Model size: ~80 KB (vs. GBs for deep learning)
-  Runtime memory: < 10 MB
-  Training: **single pass** (seconds, not hours)

Remark. MoleHD matches or beats GNNs on some tasks while being orders of magnitude smaller and faster. However, it only captures *sequential* patterns in SMILES.

HDBind: Beyond SMILES

HDBind (Jones et al., 2024) addresses MoleHD's limitations by introducing **multiple encoding strategies**:

Encoding	Input	Method	Key Feature
DECFP	Molecular graph	ECFP → direct use as HV	No random projection needed
RPFP	Any feature vector	Random projection	Flexible, general purpose
HDB-MoLFormer	SMILES	LLM embedding → RP	Pre-trained knowledge
HDB-MolCLR	Graph	GNN embedding → RP	Graph SSL knowledge
HDB-Combo	SMILES + Graph	DECFP $\otimes$ MoLFormer	First fusion model

Key idea. HDBind shows that *what* you encode matters more than *how* you encode it — the choice of molecular representation drives performance.

HDBind: DECFP Encoding

The **DECFP** (Direct ECFP) encoding is remarkably simple:

$$\mathbf{h}_{\text{DECFP}} = \sigma(E_{D,r}(\text{SMILES}))$$

where:

- $E_{D,r}$ is the ECFP algorithm with `nBits = D` and `radius = r`
- σ is sign quantization

Key idea. ECFP fingerprint bits *already behave like hypervector components*:

- High-dimensional (set $D = 10,000$)
- Sparse and quasi-random
- Capture local substructure information

HDBind: DECFP Encoding

No random projection matrix needed! This makes DECFP:

- ⚡ The most **energy-efficient** encoding (0.06 J/molecule)
- 🏆 Competitive with learned representations

HDBind: Random Projection (RPFP)

For feature vectors that are not naturally high-dimensional:

$$\mathbf{z} = \mathbf{x} \cdot \mathbf{W}^\top, \quad \mathbf{h} = \sigma(\mathbf{z})$$

where:

- $\mathbf{x} \in \mathbb{R}^n$ is the input feature vector
- $\mathbf{W} \in \mathbb{R}^{D \times n}$ is a random matrix (sampled from unit sphere)
- σ is sign quantization

Theorem (Johnson-Lindenstrauss). Random projection approximately preserves pairwise distances. If two molecules are similar in input space, they will be similar in hyperspace.

HDBind: Random Projection (RPFP)

Used for: Projecting MoLFormer (768-dim) and MolCLR embeddings into D -dimensional hypervector space.

HDBind: Results Comparison

ROC-AUC on MoleculeNet (scaffold split, $D = 10,000$):

Method	BBBP	Tox21	ClinTox	HIV	BACE	SIDER
MoleHD	84.4	—	98.7	—	—	56.6
HDB-RPFP	94.8	70.8	86.3	71.8	71.3	55.2
HDB-DECFP	93.8	69.6	90.6	77.8	74.7	61.4
HDB-MoLFormer	99.2	67.3	98.8	79.2	66.8	55.4
HDB-Combo	97.4	70.1	90.7	77.4	67.0	58.8
MoLFormer-XL*	93.7	84.7	94.8	82.2	88.2	69.0

**Fine-tuned deep learning model (orders of magnitude more expensive)*

Remark. Different encodings win on different tasks — no single encoding dominates. This motivates **fusion**.

HDBind: Energy Efficiency

Method	Encode Energy (J/mol)	Device
HDB-DECFP	0.06	CPU
HDB-MoLFormer	0.39	GPU
HDB-Combo	0.50	GPU
MLP-small	0.07	GPU
MLP-large	0.07	GPU

HDBind: Energy Efficiency at Scale **At scale**

screening millions of molecules × thousands of proteins:

- HDB-DECFP on FPGA: **immediately** more efficient than all methods
- HDB-Combo on FPGA: more efficient than MLP at **10k+ protein targets**

Key idea. HDC's simple arithmetic (element-wise ops + dot products) naturally maps to energy-efficient hardware like FPGAs. This is crucial for real-world drug screening at scale.

LIT-PCBA: Real-World Virtual Screening

LIT-PCBA (Tran-Nguyen et al., 2020): A challenging benchmark with:

- 7,761 actives + 382,674 inactives (1:50 ratio)
- 15 protein targets
- Bias-minimized AVE split

HDBind results on LIT-PCBA:

- At $D \geq 10,000$: all HDBind methods **outperform MLP baselines**
- **HDB-Combo improves with increasing D** (especially $D > 10k$)
- HDB-Combo achieves **the largest published improvement** on this task

Key idea. On realistic, large-scale drug screening tasks, HDC not only is competitive but can outperform conventional ML while using a fraction of the energy.

Part IV

Information Fusion in HDC

What is Information Fusion?

Definition. Information fusion (or data fusion) is the process of combining data or representations from multiple sources to produce a more consistent, accurate, and informative representation than any individual source.

In the molecular context:

- Source 1: SMILES-based encoding → sequential patterns
- Source 2: ECFP fingerprint → graph substructures
- Source 3: LLM embedding → distributional learned features
- Source 4: Topological descriptors → 3D shape features

Goal: Combine all views into **one hypervector** that is more informative than any single view.

Why Fusion? The Complementarity Argument

Each molecular representation captures **different information**:

Representation	Captures	Misses
SMILES tokens	Sequential patterns, substrings	3D shape, global topology
ECFP	Local atom neighborhoods	Long-range interactions, 3D
MolFormer	Distributional patterns (1B+ mols)	Explicit structure, 3D
Topological (PH)	Cavities, loops, global shape	Atom-level chemistry

Key idea. No single view captures everything. Fusion exploits **complementarity** — combining views that encode *non-overlapping* information.

Multi-View Learning: The ML Perspective

Multi-view learning is a well-studied paradigm in ML:

Core principle: If data can be described from multiple independent "views", using all views should improve performance.

Three strategies:

1. **Early fusion:** Concatenate features, then train one model
2. **Late fusion:** Train separate models, combine predictions
3. **Intermediate fusion:** Combine learned representations mid-pipeline

Key idea. HDC fusion is a form of **intermediate fusion** — we combine *encoded representations* (hypervectors) before classification. The key advantage is that HDC has native algebraic operations designed for composition.

Fusion Strategy 1: Binding (Hadamard Product)

$$\mathbf{h}_{\text{fused}} = \mathbf{h}_A \otimes \mathbf{h}_B = \langle a_1b_1, a_2b_2, \dots, a_Db_D \rangle$$

Semantic meaning: "A combined with B forms a new joint entity"

-  Creates a representation **orthogonal to both inputs**
-  Distributes over bundling
-  Invertible (can recover one view given the other)
-  Loses individual view similarity (cannot query "is this similar to view A?")

This is what HDB-Combo uses:

$$\mathbf{h}_{\text{Combo}} = \mathbf{h}_{\text{DECFP}} \otimes \mathbf{h}_{\text{MoLFormer}}$$

Remark. Binding creates a fundamentally new representation. It works best when both views are reliably informative.

Fusion Strategy 2: Bundling (Superposition)

$$\mathbf{h}_{\text{fused}} = \sigma(\mathbf{h}_A + \mathbf{h}_B)$$

Semantic meaning: "A and B are both part of this entity"

Properties for fusion:

-  Result is **similar to all inputs**: preserves individual view information
-  Robust: a noisy or uninformative view causes less damage
-  Not invertible: cannot separate views after fusion
-  Capacity limit: $\sim\sqrt{D}$ views maximum

Key idea. Bundling is gentler than binding — it **preserves** each view's identity in the fused representation. If one view is noisy, bundling degrades more gracefully than binding.

Binding vs. Bundling for Fusion: Critical Comparison

Property	Binding ($\otimes$)	Bundling ($\oplus$)
Similarity to inputs	Orthogonal (dissimilar)	Similar to all inputs
Information behavior	Creates new joint entity	Creates superposition
Noise sensitivity	High (bad view corrupts result)	Low (graceful degradation)
Capacity	Unlimited compositions	$\sim \sqrt{D}$ vectors
Invertibility	Yes (self-inverse)	No
Best for	Complementary, reliable views	Redundant or noisy views

Key idea. The choice of fusion operation has **semantic and practical consequences**. There is no universally best choice — it depends on view quality and complementarity. **This is an open research question.**

HDB-Combo: The First Molecular Fusion Model

HDBind introduced HDB-Combo (Jones et al., 2024):

$$\mathbf{h}_{\text{Combo}} = \mathbf{h}_{\text{DECFP}} \otimes \mathbf{h}_{\text{MoLFormer}}$$

What it combines:

- ECFP substructure information (graph view)
- MoLFormer learned features (LLM view)

HDB-Combo: The First Molecular Fusion Model

Results:

-  Generally high performance across 5/6 MoleculeNet tasks
-  Best overall improvement on LIT-PCBA at $D > 10k$
-  **Not best on any single task** — each individual view beats it somewhere

Remark. HDB-Combo shows fusion *works*, but its simple binding approach may not be optimal. This gap is a key research opportunity.

Why HDB-Combo Falls Short

The problem: Simple Hadamard product assumes both views are equally reliable and informative.

But in practice:

- HDB-MoLFormer dominates on BBBP (99.2 vs. 97.4 for Combo)
- HDB-DECFP dominates on SIDER (61.4 vs. 58.8 for Combo)
- Binding with a weaker view can **dilute** a stronger view

The Path Forward for Fusion

What's needed:

1. **Weighted fusion:** Give more importance to the better view
2. **Adaptive fusion:** Change weights per task or per molecule
3. **More views:** Add topological, 3D, or other encodings
4. **Better operators:** Investigate bundling, attention, or hybrid strategies

Key idea. The gap between HDB-Combo and per-task best encoders is the **research space** for Thesis 1.

Weighted Bundling

Instead of equal-weight bundling, assign **learned importance scores**:

$$\mathbf{h}_{\text{fused}} = \sigma \left(\sum_{i=1}^N w_i \cdot \mathbf{h}_i \right)$$

How to determine weights w_i :

Method	Type	Complexity
Uniform ($w_i = 1/N$)	Fixed	None
Performance-based	Fixed	Low
Validation-tuned	Fixed	Medium
Learned (gradient)	Trainable	Medium
Uncertainty-based	Adaptive	Medium
Attention-based	Per-sample adaptive	Higher

Ensemble HDC

An alternative to representation-level fusion is **prediction-level ensemble**:

Approach: Train separate HDC classifiers, combine predictions

Techniques:

- **Majority voting:** Each model votes for a class
- **Confidence weighting:** Weight each model's vote by its confidence η
- **Bagging:** Multiple HDC models with different encodings, dimensions, or quantization

Ensemble HDC: Pros and Cons

Advantages:

- No need to choose a fusion operator
- Each model can use the optimal encoding for its data

Disadvantages:

- N models = $N \times$ inference cost
- No shared representation (cannot query the fused HV)

Remark. Ensemble HDC is an important baseline for fusion research — it's the "upper bound" of what independent views can achieve.

GraphHD: Encoding Graphs as Hypervectors

GraphHD (Nunes et al., 2022, Vazquez & Sica, 2025) provides a principled way to encode entire graphs:

Algorithm:

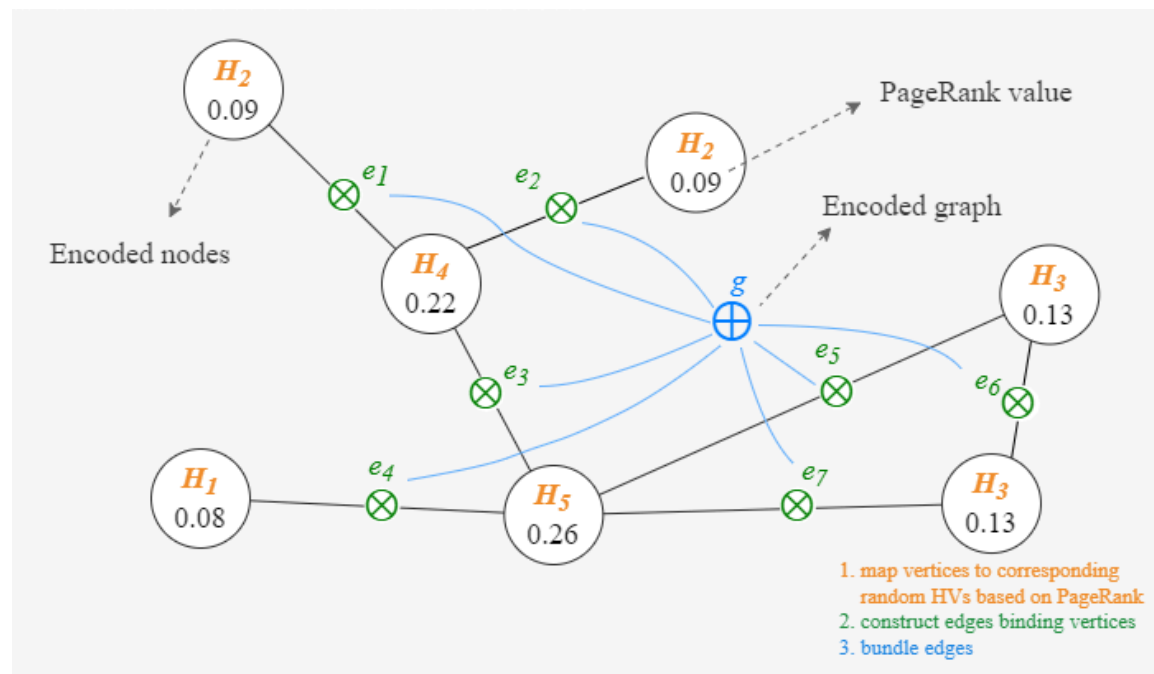
1. Compute **PageRank centrality** for each node
2. Discretize centrality into bins, each assigned a random HV
3. Create edge HVs by **binding** node HVs:

$$\mathbf{h}_{\text{edge}} = \mathbf{h}_u \otimes \mathbf{h}_v$$

4. Create graph HV by **bundling** all edge HVs:

$$\mathbf{h}_{\text{graph}} = \bigoplus_{(u,v) \in E} \mathbf{h}_u \otimes \mathbf{h}_v$$

GraphHD: Encoding Graphs as Hypervectors



Key idea. GraphHD shows that molecular *graphs* can be directly encoded into hypervectors — preserving structural information differently from ECFP or SMILES-based methods.

MultimodalHD: Attentive Fusion

MultimodalHD (2023-2024) introduces **attention-based fusion** for HDC:

Problem: Fixed-weight bundling ignores varying reliability of modalities.

Solution: Learn per-sample attention weights:

1. Each modality encoded via standard HDC encoder
2. An attention module computes cross-modal correlations
3. Weighted fusion produces the final representation

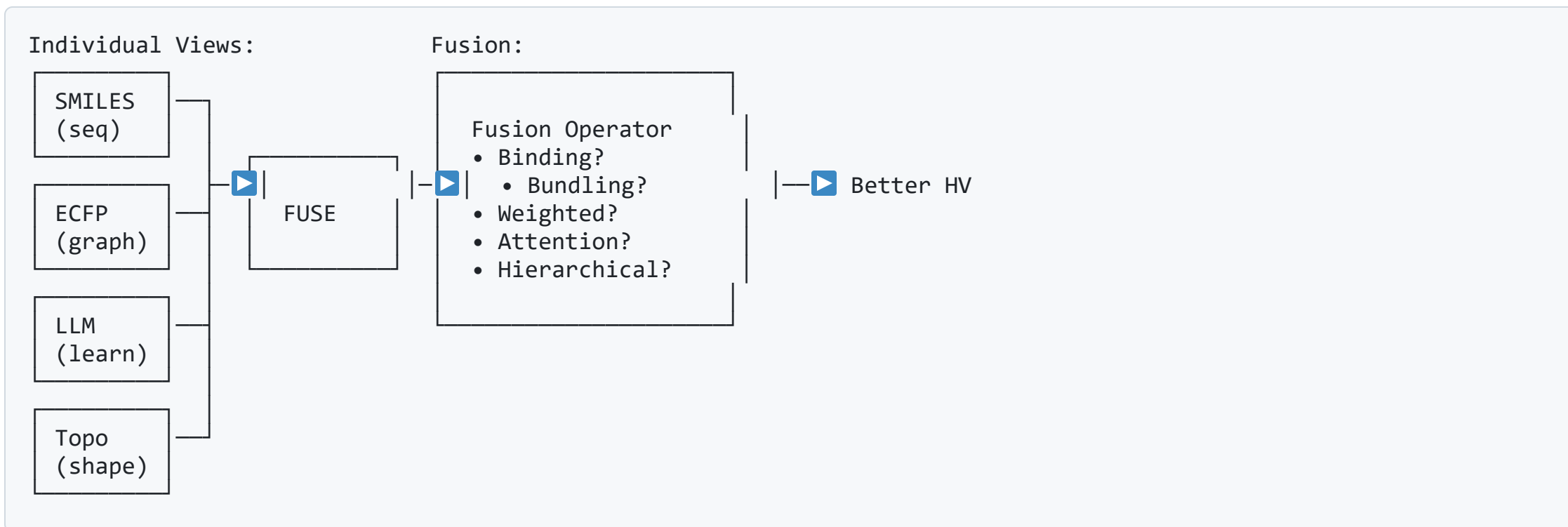
MultimodalHD: Attentive Fusion

Results (on sensor data):

- 2-8× training speedup over RNN/LSTM
- Improved accuracy over fixed bundling

Key idea. MultimodalHD demonstrates that **attention-based fusion in HDC** improves over both binding and uniform bundling — directly relevant to Thesis 2.

The Fusion Opportunity: Summary



Key idea. The research question is: which fusion operator, applied to which views, with what weighting, yields the best molecular property prediction?

Part V

Attention Mechanisms

Attention: The Intuition

Human analogy: When reading, you don't give equal focus to every word — you attend selectively to what matters.

ML analogy: When combining information from multiple sources, not all sources are equally relevant for a given input.

Key idea. Attention is a mechanism that computes **dynamic, input-dependent weights** that determine how much each piece of information contributes to the output.

Contrast with fixed weights:

- Fixed weights: $w_1 = 0.5, w_2 = 0.5$ (same for all inputs)
- Attention weights: $\alpha_1 = f(\text{input}), \alpha_2 = g(\text{input})$ (varies per input)

Dot-Product Attention: The Basics

Given a **query** $\mathbf{q}$ and a set of **keys** $\{\mathbf{k}_i\}$ with corresponding **values** $\{\mathbf{v}_i\}$:

Step 1: Compute compatibility scores:

$$e_i = \mathbf{q} \cdot \mathbf{k}_i$$

Step 2: Normalize to get attention weights:

$$\alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)} = \text{softmax}(e_i)$$

Step 3: Compute weighted sum of values:

$$\text{output} = \sum_i \alpha_i \cdot \mathbf{v}_i$$

Definition. Attention is a weighted aggregation mechanism where the weights are computed dynamically based on the compatibility between a query and a set of keys

Scaled Dot-Product Attention

The standard attention formula (Vaswani et al., 2017):

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

where:

- $Q \in \mathbb{R}^{n \times d_k}$: query matrix
- $K \in \mathbb{R}^{m \times d_k}$: key matrix
- $V \in \mathbb{R}^{m \times d_v}$: value matrix
- $\sqrt{d_k}$: scaling factor to prevent vanishing gradients

Remark. The $\sqrt{d_k}$ scaling is crucial: without it, dot products in high dimensions become very large, pushing softmax to extreme values.

Self-Attention vs. Cross-Attention

Self-attention: Query, Key, Value all come from the **same** sequence

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

- Each element "attends" to every other element in the same set
- Captures internal relationships within one modality

Cross-attention: Query from one source, Key/Value from another

$$Q = X_1W_Q, \quad K = X_2W_K, \quad V = X_2W_V$$

- Source 1 "attends" to Source 2
- Captures relationships **between** different modalities

Key idea. For fusion in HDC, **cross-attention** is key: each molecular view attending to the others to determine its optimal weight.

Multi-Head Attention

Instead of one attention function, use h parallel "heads":

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Why multiple heads?

- Each head can focus on **different aspects** of the input
- Head 1 might attend to local chemistry
- Head 2 might attend to global shape
- Head 3 might attend to functional groups

Remark. Multi-head attention allows the model to jointly attend to information from

Attention in the HDC Context

How does attention apply to hypervector fusion?

Setting: We have N view hypervectors $\{\mathbf{h}_1, \dots, \mathbf{h}_N\}$ for one molecule.

Goal: Compute per-view attention weights $\{\alpha_1, \dots, \alpha_N\}$.

Approach 1 — Query-based attention:

$$e_i = \frac{\mathbf{q} \cdot \mathbf{h}_i}{\sqrt{D}}, \quad \alpha_i = \text{softmax}(e_i)$$

$$\mathbf{h}_{\text{fused}} = \sigma \left(\sum_{i=1}^N \alpha_i \cdot \mathbf{h}_i \right)$$

where $\mathbf{q}$ is a learnable query vector.

Key idea. The attention weights α_i are computed **per molecule** — different molecules

Cross-View Attention for Fusion

Approach 2 — Cross-view attention (more expressive):

Each view hypervector attends to all other views:

$$e_{ij} = \frac{\mathbf{h}_i \cdot \mathbf{h}_j}{\sqrt{D}}, \quad \alpha_{ij} = \text{softmax}_j(e_{ij})$$

$$\tilde{\mathbf{h}}_i = \sum_{j=1}^N \alpha_{ij} \cdot \mathbf{h}_j$$

Then combine the attended views:

$$\mathbf{h}_{\text{fused}} = \sigma \left(\sum_{i=1}^N \tilde{\mathbf{h}}_i \right)$$

Key idea. Cross-view attention lets each view "look at" the others and selectively

Attention vs. Fixed Fusion: Trade-offs

Aspect	Fixed Fusion ($\otimes$ or $\oplus$)	Attention-Based Fusion
Weights	Fixed for all molecules	Dynamic, per-molecule
Parameters	None (0 learnable params)	Projection matrices (W_Q, W_K, W_V)
Training	No gradient descent needed	Requires gradient-based training
Adaptivity	None	Task-adaptive, input-adaptive
Efficiency	 Ultra-fast	Slightly slower (matrix ops)
Interpretability	Implicit	Explicit (can inspect weights)

Key idea. Attention adds a small amount of complexity (learnable projections) but gains significant flexibility. **The key question for Thesis 2:** Does this flexibility justify the added cost in the HDC setting?

The Efficiency Question

A critical concern: HDC's main advantage is simplicity and efficiency. Does attention destroy this?

Analysis of added cost:

- Attention requires: N dot products + softmax + weighted sum
- For $N = 4$ views, $D = 10,000$: $\sim 40,000$ multiplications
- Compare to: HDC inference dot product = 10,000 multiplications per class

Conclusion: Attention adds $O(N \cdot D)$ cost — linear in both the number of views and the dimensionality.

For $N = 4$: roughly **4× the cost of basic inference**. This is acceptable if accuracy improves significantly.

Part VI

Topological Data Analysis for Molecules

What is Topology?

Definition. **Topology** is the branch of mathematics that studies properties of shapes that are preserved under continuous deformations (stretching, bending — but not tearing or gluing).

Topological invariants — properties that don't change:

- Number of **connected components** (how many pieces?)
- Number of **loops/holes** (does it have a cavity?)
- Number of **voids** (enclosed 3D spaces?)

Why it matters for molecules:

- A molecule's **3D shape** determines its biological activity
- Binding pockets, cavities, and ring systems are topological features

Simplicial Complexes: Building Blocks

Definition. A simplex is a generalization of a triangle to arbitrary dimensions:

- 0-simplex: a point (vertex)
- 1-simplex: a line segment (edge)
- 2-simplex: a filled triangle (face)
- 3-simplex: a filled tetrahedron

A **simplicial complex** is a collection of simplices glued together along faces.

For molecules: place a ball of radius r around each atom. As r grows:

- Balls start overlapping $\rightarrow$ edges form
- Triples overlap $\rightarrow$ triangles form
- Topological features appear and disappear

Persistent Homology: The Key Tool

Definition. Persistent Homology (PH) tracks how topological features (components, loops, cavities) appear and disappear as a parameter (radius) increases.

Filtration: A nested sequence of complexes:

$$\emptyset = K_0 \subseteq K_1 \subseteq K_2 \subseteq \dots \subseteq K_n$$

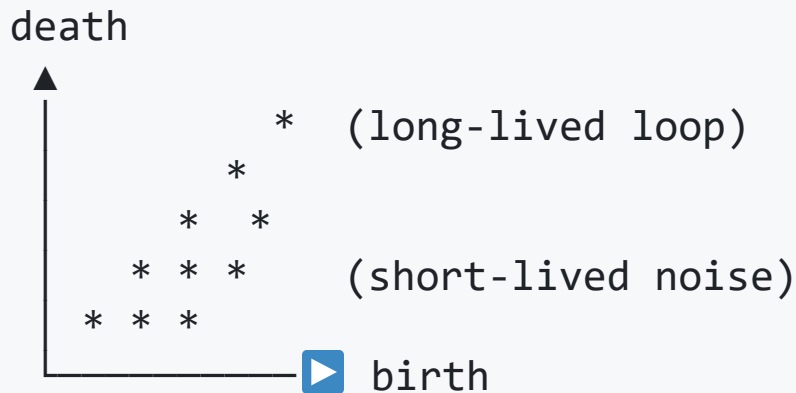
For each feature:

- **Birth** b : the radius at which the feature first appears
- **Death** d : the radius at which the feature disappears
- **Persistence**: $d - b$ (longer = more significant)

Key idea. Features with high persistence represent **real structural patterns**. Short-lived features are often noise.

Persistence Diagrams

A persistence diagram is a 2D scatter plot of (birth, death) pairs:



Three sets of features (homology dimensions):

- H_0 (dim 0): Connected components
- H_1 (dim 1): Loops and rings
- H_2 (dim 2): Cavities and voids

What Topology Captures for Molecules

Homology	Chemical Meaning	Drug Discovery Relevance
H_0	Atom clusters, fragments	Molecular connectivity
H_1	Rings, aromatic systems	Pharmacophore patterns
H_2	Cavities, binding pockets	Binding site geometry

Key idea. H_2 features (cavities and pockets) are **uniquely captured by topology**. No other molecular representation — SMILES, ECFP, language model — can directly encode this information.

This is why topological features are uniquely complementary for fusion.

Vectorization: From Diagrams to Vectors

Persistence diagrams cannot be used directly as ML features. We need **vectorization**:

1. Persistence Images (Adams et al., 2017):

- Place Gaussian kernel at each (birth, death) point
- Weight by persistence (longer = higher weight)
- Discretize on a grid → fixed-length vector

2. Persistence Landscapes (Bubenik, 2015):

- Convert diagram to piecewise-linear functions
- Sample at regular intervals → fixed-length vector

3. Summary Statistics:

- Mean/max persistence, entropy, number of features

Topological Hypervector Encoding (Novel Contribution)

This encoding has NOT been previously proposed in the HDC literature.

Pipeline:

1. **Conformer generation:** SMILES $\rightarrow$ 3D coordinates (RDKit, MMFF94)
2. **Persistent homology:** Compute H_0, H_1, H_2 persistence diagrams
3. **Vectorization:** Convert diagrams to persistence images
4. **Random projection:** Project to D -dimensional hyperspace

$$\mathbf{h}_{\text{topo}} = \sigma (\mathbf{x}_{\text{PI}} \cdot W^{\top})$$

where $\mathbf{x}_{\text{PI}}$ is the concatenated persistence image vector.

Key idea: This creates a new type of molecular hypervector that captures 3D shape,

Element-Specific Persistent Homology

Advanced variant: Separate filtrations by atom type:

- **C-C filtration:** Topology of carbon scaffolds
- **C-N filtration:** Topology of nitrogen-containing regions
- **C-O filtration:** Topology of oxygen-containing regions

Each produces its own persistence diagram → its own features.

Why this helps:

- Different atom pairs define different chemical environments
- A cavity lined with nitrogen atoms (potential hydrogen bond acceptors) is functionally different from a carbon-only cavity

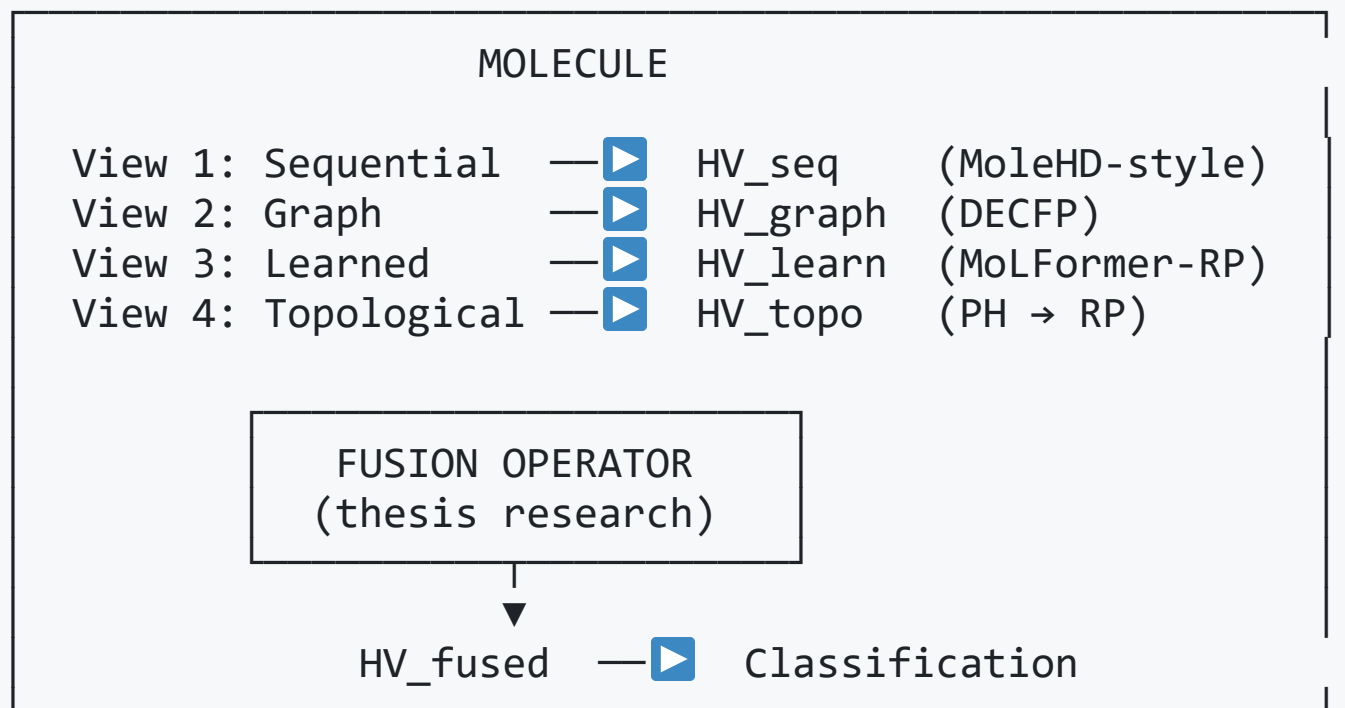
Remark. Element-specific PH has shown improvements in non-HDC molecular ML (Wei

Part VII

Proposed Research Framework & Thesis Directions

Multi-View Molecular Hypervector Fusion (MVMHF)

We propose a unified framework combining 4 complementary views:



Key idea. The framework is modular: views and fusion strategies are interchangeable,

Six Fusion Strategies to Investigate

Strategy	Formula	Type
A: Naive Bundling	$\sigma(\sum_i \mathbf{h}_i)$	Fixed
B: Pairwise Bind + Bundle	$\sigma((\mathbf{h}_1 \otimes \mathbf{h}_2) + (\mathbf{h}_3 \otimes \mathbf{h}_4))$	Fixed
C: Full Binding	$\mathbf{h}_1 \otimes \mathbf{h}_2 \otimes \mathbf{h}_3 \otimes \mathbf{h}_4$	Fixed
D: Weighted Bundling	$\sigma(\sum_i w_i \mathbf{h}_i)$	Learned
E: Attentive Fusion	$\sigma(\sum_i \alpha_i \mathbf{h}_i), \alpha_i = f(\mathbf{h}_i)$	Per-sample
F: Hierarchical	$\sigma(w_1(\mathbf{h}_{\text{seq}} \otimes \mathbf{h}_{\text{graph}}) + w_2(\mathbf{h}_{\text{learn}} \otimes \mathbf{h}_{\text{topo}}))$	Hybrid

Key idea. The strategies range from zero-parameter (A, B, C) to learnable (D, E, F), allowing us to measure the value of added complexity.

Experimental Design

Datasets: MoleculeNet (BBBP, Tox21, ClinTox, HIV, BACE, SIDER) + LIT-PCBA

Protocol:

- Scaffold split (primary), random split (control)
- $D \in \{1k, 5k, 10k, 50k, 100k\}$
- 10 random seeds per configuration
- Metrics: ROC-AUC, ER-1% enrichment

Experimental Design: Ablation Studies

Ablation studies:

Experiment	Purpose
Single view vs. pair vs. all 4	Value of each view
Strategy A vs. B vs. C vs. D vs. E vs. F	Best fusion operator
Remove one view at a time	Most critical view
H_0 vs. H_1 vs. H_2 separately	Most useful topological dimension
Effect of D on fusion	Optimal dimensionality

Thesis 1: HDC Fusion for Molecular Properties

Title: *Multi-View Hypervector Fusion for Molecular Property Prediction*

Research questions:

1. Does fusing ECFP + topological views improve over ECFP alone?
2. Which fusion operator (binding vs. bundling vs. weighted) is optimal?
3. Does adding more views always help, or is there diminishing return?

Thesis 2: Scope

Scope:

- Implement MVMHF framework with Views 2 (ECFP) + 4 (Topological)
- Evaluate Strategies A, B, C, D on MoleculeNet + LIT-PCBA
- Compare against HDB-DECFP and HDB-Combo baselines

Expected contribution: First systematic study of multi-view fusion operators in molecular HDC, including the novel topological encoding.

Thesis 2: Attention Mechanism for HDC Properties Prediction

Title: *Attention-Based Fusion for Hyperdimensional Molecular Classification*

Research questions:

1. Does attention-based fusion (Strategy E) outperform fixed fusion?
2. Can attention weights reveal which views matter for which tasks?
3. What is the efficiency/accuracy trade-off of adding attention to HDC?

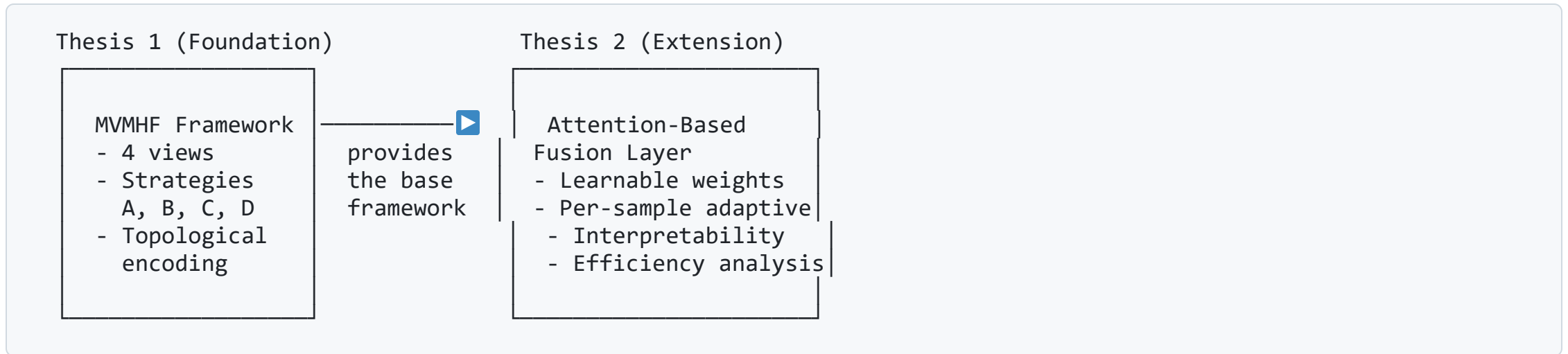
Thesis 2: Scope

Scope:

- Implement attention fusion layer (query-based + cross-view)
- Compare Strategy E against A, B, C, D on same benchmarks
- Analyze learned attention weights for interpretability
- Measure energy consumption overhead of attention

Expected contribution: First application of attention-based fusion in molecular HDC, with analysis of the efficiency-accuracy trade-off.

How the Two Theses Connect



Thesis 1 builds the multi-view encoding and tests fixed fusion operators.

Thesis 2 takes the best views from Thesis 1 and adds learnable attention.

Key idea. The theses are complementary: Thesis 1 answers "what to fuse?" and Thesis 2 answers "how to best fuse it?"

Open Research Questions

Fundamental:

- Is binding or bundling better for multi-view molecular HDC?
- Does topological information (H_2) significantly help binding affinity tasks?
- What dimensionality D is optimal for fused representations?

Methodological:

- How does conformer quality affect topological encoding?
- Can attention be kept lightweight enough for HDC efficiency?
- Is element-specific PH more informative than standard PH?

Open Research Questions: Theory

Theoretical:

- How many views can be fused before representation degrades?
- Under what conditions does fusion provably improve over single-view?

Key idea. These questions define a rich research program extending well beyond the two theses.

Summary

What we covered:

1.  **Molecular representations:** SMILES, ECFP, molecular graphs, 3D, learned (MoLFormer)
2.  **HDC foundations:** Hypervectors, binding, bundling, permutation, encoding, training, inference
3.  **HDC for molecules:** MoleHD (SMILES-based), HDBind (multi-encoding, fusion)
4.  **Information fusion:** Binding vs. bundling, weighted fusion, ensemble, MultimodalHD

Summary

What we covered:

5.  **Attention mechanisms:** Dot-product, scaled, self/cross, multi-head, attention in HDC
6.  **Topological data analysis:** Persistent homology, persistence diagrams, vectorization
7.  **Thesis directions:** MVMHF framework, 6 strategies, experimental design

Key References

1. Ma et al. (2022). *MoleHD: Efficient Drug Discovery using Brain Inspired HDC*. IEEE/ACM ICCAD.
2. Jones et al. (2024). *HDBind: Encoding of molecular structure with HD binary representations*. Sci. Reports.
3. Kanerva (2009). *Hyperdimensional Computing: An Introduction*. Cognitive Computation.
4. Kleyko et al. (2022-2023). *A Survey on HDC aka VSA (Parts I & II)*. ACM Computing Surveys.
5. Vaswani et al. (2017). *Attention Is All You Need*. NeurIPS.
6. Adams et al. (2017). *Persistence Images: A Stable Vector Representation of PH*. JMLR.

7. Wu et al. (2018). *MoleculeNet: A benchmark for molecular ML*. Chemical Science.
8. Ross et al. (2022). *Large-scale chemical language representations*. Nature Machine Intelligence.
9. MultimodalHD (2023-2024). *Attentive Multimodal Fusion for HDC*. DATE/IPSN.

Thank You

Questions?

Next steps: Set up development environment (Python, RDKit, PyTorch) and begin implementing the MVMHF encoding pipeline.